| SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE | | DEPARTMENT OF COMPUTER SCIENCE ENGINEERING | |
|---|---|---|---|
| Program Name: B. Tech | Assignment Type: Lab | | Academic Year: 2025-2026 |
| **Course Coordinator Name** | | Dr. Rishabh Mittal | |
| **Instructor(s) Name** | | | |
| **Course Code** | 23CS002PC304 | **Course Title** | AI Assisted Coding |
| **Year/Sem** | III/II | **Regulation** | R23 |
| **Date and Day of Assignment** | Week4 - Tuesday | **Time(s)** | |
| **Duration** | 2 Hours | **Applicable to Batches** | 23CSBTB01 To 23CSBTB52 |
| **Assignment Number: 8.2 (Present assignment number) / 24 (Total number of assignments)** | | | |

# ASSIGNMENT-8.2

**Name:** **Daniel Anthonello**
**HT. No:** 2303A51438
**Batch:** **7**

---

## Task 1: Test-Driven Development for Even/Odd Number Validator

**Scenario:** Use AI tools to first generate test cases for a function **is_even(n)** and then implement the function so that it satisfies all generated tests. The function handles integers including zero, negative numbers, and large integers.

**Prompt:**
Generate comprehensive unittest test cases for an is_even(n) function in Python that checks whether a number is even. Include edge cases for positive even/odd numbers, zero, negative numbers, large integers, and invalid inputs (non-integers).

**Code:**

```
import unittest

def is_even(n):
if not isinstance(n, int):
raise TypeError("Input must be an integer")
return n % 2 == 0

class TestIsEven(unittest.TestCase):
def test_positive_even(self):
self.assertTrue(is_even(2))
def test_positive_odd(self):
self.assertFalse(is_even(7))
def test_zero(self):
self.assertTrue(is_even(0))
def test_negative_even(self):
self.assertTrue(is_even(-4))
def test_negative_odd(self):
self.assertFalse(is_even(9))
def test_large_integer(self):
self.assertTrue(is_even(1000000000))
def test_invalid_input(self):
with self.assertRaises(TypeError):
is_even("2")

if __name__== "__main__":
unittest.main()
```

**Screenshot / Output:**

```
day5.py > TestIsEven
import unittest
def is_even(n):
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")
    return n % 2 == 0
class TestIsEven(unittest.TestCase):
    def test_positive_even(self):
        self.assertTrue(is_even(2))

    def test_positive_odd(self):
        self.assertFalse(is_even(7))

    def test_zero(self):
        self.assertTrue(is_even(0))

    def test_negative_even(self):
        self.assertTrue(is_even(-4))

    def test_negative_odd(self):
        self.assertFalse(is_even(9))

    def test_large_integer(self):
        self.assertTrue(is_even(1000000000))

    def test_invalid_input(self):
        with self.assertRaises(TypeError):
            is_even("2")
if __name__ == "__main__":
    unittest.main()
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
mohammadmuneerahmed@Muneers-MacBook-Air training2.py % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
.......
----------------------------------------------------------------------
Ran 7 tests in 0.000s

OK
mohammadmuneerahmed@Muneers-MacBook-Air training2.py %
```

*Task 1: is_even() – All 7 tests passed (OK)*

**Result:**

The is_even() function correctly handles all edge cases. The unittest framework ran 7 tests in 0.000s with all tests passing (OK). The function properly raises a TypeError for non-integer inputs.

**Observation:**

The AI-generated test cases comprehensively covered positive even/odd numbers, zero, negative numbers, large integers, and invalid string inputs. Using isinstance() for type validation ensures robust input checking. The modulo operator (n % 2 == 0) is an efficient and Pythonic way to test for even numbers.

## Task 2: Test-Driven Development for String Case Converter

**Scenario:** Ask AI to generate test cases for two functions: **to_uppercase(text)** and **to_lowercase(text)**. Both functions must handle empty strings, mixed-case input, and invalid inputs such as numbers or None.

**Prompt:**

Generate unittest test cases for to_uppercase(text) and to_lowercase(text) functions. Include tests for normal strings, empty strings, mixed-case input, None values, and numeric inputs.

**Code:**

```
import unittest

def to_uppercase(text):
if text is None:
raise ValueError("Input cannot be None")
if not isinstance(text, str):
raise TypeError("Input must be a string")
return text.upper()

def to_lowercase(text):
if text is None:
raise ValueError("Input cannot be None")
if not isinstance(text, str):
raise TypeError("Input must be a string")
return text.lower()

class TestStringCaseConverter(unittest.TestCase):
def test_uppercase(self):
self.assertEqual(to_uppercase("ai coding"), "AI CODING")
def test_lowercase(self):
self.assertEqual(to_lowercase("TEST"), "test")
def test_empty_string(self):
self.assertEqual(to_uppercase(""), "")
self.assertEqual(to_lowercase(""), "")
def test_invalid_none(self):
with self.assertRaises(ValueError):
to_lowercase(None)
def test_invalid_number(self):
with self.assertRaises(TypeError):
to_uppercase(123)

if __name__== "__main__":
unittest.main()
```

**Screenshot / Output:**

```
day5.py > ...
import unittest
def to_uppercase(text):
    if text is None:
        raise ValueError("Input cannot be None")
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.upper()
def to_lowercase(text):
    if text is None:
        raise ValueError("Input cannot be None")
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.lower()
class TestStringCaseConverter(unittest.TestCase):
    def test_uppercase(self):
        self.assertEqual(to_uppercase("ai coding"), "AI CODING")
    def test_lowercase(self):
        self.assertEqual(to_lowercase("TEST"), "test")
    def test_empty_string(self):
        self.assertEqual(to_uppercase(""), "")
        self.assertEqual(to_lowercase(""), "")
    def test_invalid_none(self):
        with self.assertRaises(ValueError):
            to_lowercase(None)
    def test_invalid_number(self):
        with self.assertRaises(TypeError):
            to_uppercase(123)
if __name__ == "__main__":
    unittest.main()
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
mohammadmuneerahmed@Muneers-MacBook-Air training2.py % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
.....
----------------------------------------------------------------------
Ran 5 tests in 0.000s

OK
mohammadmuneerahmed@Muneers-MacBook-Air training2.py %
```

*Task 2: String Case Converter – 5 tests passed (OK)*

**Result:**
Both string conversion functions work correctly. The unittest framework ran 5 tests in 0.000s with all tests passing (OK). The functions raise ValueError for None and TypeError for non-string inputs.

**Observation:**
Separating None and type validation into distinct checks allows more specific error handling. The AI correctly identified that both ValueError (for None) and TypeError (for wrong types) are needed. Empty strings are handled gracefully by Python's built-in upper() and lower() methods.

## Task 3: Test-Driven Development for List Sum Calculator

**Scenario:** Use AI to generate test cases for a function **sum_list(numbers)** that calculates the sum of list elements, handling empty lists, negative numbers, and ignoring non-numeric values.

**Prompt:**

Generate unittest test cases for a sum_list(numbers) function. Include tests for normal lists, empty lists, negative numbers, mixed-type lists (with strings), and non-list input.

**Code:**

```
import unittest

def sum_list(numbers):
if not isinstance(numbers, list):
raise TypeError("Input must be a list")
total = 0
for item in numbers:
if isinstance(item, (int, float)):
total += item
return total

class TestSumList(unittest.TestCase):
def test_normal_list(self):
self.assertEqual(sum_list([1, 2, 3]), 6)
def test_empty_list(self):
self.assertEqual(sum_list([]), 0)
def test_negative_numbers(self):
self.assertEqual(sum_list([-1, 5, -4]), 0)
def test_mixed_values(self):
self.assertEqual(sum_list([2, "a", 3]), 5)
def test_invalid_input(self):
with self.assertRaises(TypeError):
sum_list("123")

if __name__== "__main__":
unittest.main()
```

**Screenshot / Output:**

```
day5.py > TestSumList > test_normal_list
import unittest
def sum_list(numbers):
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
    total = 0
    for item in numbers:
        if isinstance(item, (int, float)):
            total += item
    return total
class TestSumList(unittest.TestCase):
    def test_normal_list(self):
        self.assertEqual(sum_list([1, 2, 3]), 6)
    def test_empty_list(self):
        self.assertEqual(sum_list([]), 0)

    def test_negative_numbers(self):
        self.assertEqual(sum_list([-1, 5, -4]), 0)

    def test_mixed_values(self):
        self.assertEqual(sum_list([2, "a", 3]), 5)

    def test_invalid_input(self):
        with self.assertRaises(TypeError):
            sum_list("123")


if __name__ == "__main__":
    unittest.main()
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
mohammadmuneerahmed@Muneers-MacBook-Air training2.py % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
.....
----------------------------------------------------------------------
Ran 5 tests in 0.000s

OK
mohammadmuneerahmed@Muneers-MacBook-Air training2.py %
```

*Task 3: sum_list() – 5 tests passed (OK)*

**Result:**
The sum_list() function correctly handles all scenarios. The unittest framework ran 5 tests in 0.000s with all tests passing (OK). Non-numeric values in the list are silently ignored.

**Observation:**
Using isinstance(item, (int, float)) for selective summation is a clean approach to handle mixed-type lists. The function raises a TypeError for non-list inputs, preventing misuse. AI-generated tests effectively covered boundary conditions like empty lists and all-negative values.

## Task 4: Test Cases for Student Result Class

**Scenario:** Generate test cases for a **StudentResult** class with methods add_marks(mark), calculate_average(), and get_result(). Marks must be between 0 and 100, and average >= 40 is a Pass.

**Prompt:**

Generate unittest test cases for a StudentResult class. Include test cases for passing/failing averages, invalid marks (negative or above 100), and empty mark lists.

**Code:**

```
import unittest

class StudentResult:
def __init__(self):
self.marks = []

def add_marks(self, mark):
if not isinstance(mark, (int, float)):
raise TypeError("Marks must be numeric")
if mark < 0 or mark > 100:
raise ValueError("Marks must be between 0 and 100")
self.marks.append(mark)

def calculate_average(self):
if not self.marks:
return 0
return sum(self.marks) / len(self.marks)

def get_result(self):
average = self.calculate_average()
return "Pass" if average >= 40 else "Fail"

class TestStudentResult(unittest.TestCase):
def test_pass_case(self):
student = StudentResult()
student.add_marks(60)
student.add_marks(70)
student.add_marks(80)
self.assertEqual(student.calculate_average(), 70)
self.assertEqual(student.get_result(), "Pass")

def test_fail_case(self):
student = StudentResult()
student.add_marks(30)
student.add_marks(35)
student.add_marks(40)
self.assertEqual(student.calculate_average(), 35)
self.assertEqual(student.get_result(), "Fail")

def test_invalid_marks(self):
student = StudentResult()
with self.assertRaises(ValueError):
student.add_marks(-10)

if __name__== "__main__":
unittest.main()
```

**Screenshot / Output:**

```python
import unittest

class StudentResult:
    def __init__(self):
        self.marks = []

    def add_marks(self, mark):
        if not isinstance(mark, (int, float)):
            raise TypeError("Marks must be numeric")
        if mark < 0 or mark > 100:
            raise ValueError("Marks must be between 0 and 100")
        self.marks.append(mark)

    def calculate_average(self):
        if not self.marks:
            return 0
        return sum(self.marks) / len(self.marks)

    def get_result(self):
        average = self.calculate_average()
        return "Pass" if average >= 40 else "Fail"


class TestStudentResult(unittest.TestCase):

    def test_pass_case(self):
        student = StudentResult()
        student.add_marks(60)
        student.add_marks(70)
        student.add_marks(80)
        self.assertEqual(student.calculate_average(), 70)
        self.assertEqual(student.get_result(), "Pass")

    def test_fail_case(self):
        student = StudentResult()
        student.add_marks(30)
        student.add_marks(35)
        student.add_marks(40)
        self.assertEqual(student.calculate_average(), 35)
        self.assertEqual(student.get_result(), "Fail")

    def test_invalid_marks(self):
        student = StudentResult()
        with self.assertRaises(ValueError):
            student.add_marks(-10)


if __name__ == "__main__":
    unittest.main()
```

```
python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
mohammadmuneerahmed@Muneers-MacBook-Air training2.py % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
...
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
mohammadmuneerahmed@Muneers-MacBook-Air training2.py %
```

*Task 4: StudentResult class – 3 tests passed (OK)*

**Result:**

The StudentResult class correctly calculates averages and determines pass/fail status. The unittest framework ran 3 tests in 0.000s with all tests passing (OK). Invalid marks correctly raise ValueError.

**Observation:**

The class-based approach with separate methods for adding marks, calculating average, and determining results provides clean separation of concerns. The AI correctly identified test scenarios for both pass (average=70) and fail (average=35) cases, along with boundary validation. This demonstrates TDD applied effectively to object-oriented design.

## Task 5: Test-Driven Development for Username Validator

**Scenario:** Use AI to generate test cases for an **is_valid_username(username)** function. A valid username must be at least 5 characters, contain no spaces, and consist only of alphanumeric characters.

**Prompt:**

Generate unittest test cases for a username validation function. Test valid usernames, usernames that are too short, usernames with spaces, usernames with special characters, and non-string inputs.

**Code:**

```
import unittest
import re

def is_valid_username(username):
if not isinstance(username, str):
raise TypeError("Username must be a string")
if len(username) < 5:
return False
if " " in username:
return False
if not username.isalnum():
return False
return True

class TestUsernameValidator(unittest.TestCase):
def test_valid_username(self):
self.assertTrue(is_valid_username("user01"))
def test_too_short(self):
self.assertFalse(is_valid_username("ai"))
def test_contains_space(self):
self.assertFalse(is_valid_username("user name"))
def test_special_character(self):
self.assertFalse(is_valid_username("user@123"))
def test_invalid_type(self):
with self.assertRaises(TypeError):
is_valid_username(12345)

if __name__== "__main__":
unittest.main()
```

**Screenshot / Output:**

```
day5.py > ...
import unittest
import re

def is_valid_username(username):
    if not isinstance(username, str):
        raise TypeError("Username must be a string")

    if len(username) < 5:
        return False

    if " " in username:
        return False

    if not username.isalnum():
        return False

    return True


class TestUsernameValidator(unittest.TestCase):

    def test_valid_username(self):
        self.assertTrue(is_valid_username("user01"))

    def test_too_short(self):
        self.assertFalse(is_valid_username("ai"))

    def test_contains_space(self):
        self.assertFalse(is_valid_username("user name"))

    def test_special_character(self):
        self.assertFalse(is_valid_username("user@123"))

    def test_invalid_type(self):
        with self.assertRaises(TypeError):
            is_valid_username(12345)


if __name__ == "__main__":
    unittest.main()
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
mohammadmuneerahmed@Muneers-MacBook-Air training2.py % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
.....
----------------------------------------------------------------------
Ran 5 tests in 0.000s

OK
mohammadmuneerahmed@Muneers-MacBook-Air training2.py %
```

*Task 5: is_valid_username() – 5 tests passed (OK)*

**Result:**
The username validator correctly enforces all validation rules. The unittest framework ran 5 tests in 0.000s with all tests passing (OK). The function correctly rejects usernames that are too short, contain spaces, or have special characters.

**Observation:**
Using Python's built-in string methods (isalnum(), len()) provides a clean, readable implementation. The AI-generated test suite covered all required validation rules: minimum length, space prohibition, alphanumeric constraint, and type checking. The test cases map directly to real-world username validation requirements, demonstrating the practical value of TDD.